Design Project

Mingoo Seok

PROJECT

- This year, each student designs an FFT processor
- Target spec:
  - Memory-based architecture
  - 1024-point
  - 16-bit fixed-point input
  - Radix-2
  - Must use SRAM blocks
- RTL coding from scratch: Yes; Do not use AI

METHOD

- Input: We will supply 10,240 16-bit real-valued numbers in two formats (Q1.15 and FP32),
  - Uploaded in the coursework
- Twiddle factors: The complex-number twiddle factors in the Q1.15
  - Uploaded in the coursework
- Output: 16-bit real-valued number in the Q9.7
  - You can use any resolution intermediately

METHOD

- Input and output:
  - Load inputs and twiddle factors on SRAM row-by-row using the testbench
  - Read FFT results from SRAM row-by-row using the testbench
  - Add the multiplexed address and data input and output to SRAM for the testbench access
- In processing 10,240 inputs,
  - You should load 1,024 inputs for ten times
  - You should read out 1,024 outputs for ten times
  - These cycles and power consumption should be counted in your metrics

METRICS

- Maximum clock frequency: [MHz]
  - PT with Innovus-gen. sdf annotation
- Throughput: [MS/s]
  - PT with Innovus-gen. sdf annotation
- Energy efficiency: [pJ/S]
  - PT with the Innovus-gen. sdf and QS-gen. vcd annotations
- Area: [mm^2]
  - Based on the LVS and DRC-clean layout in Cadence Virtuoso

METRICS

- Compute density: [MS/s/mm^2]
  - Based on the above throughput and area metrics
- Accuracy NRMSE
  - NRMSE compared to MATLAB results in 32-bit floating-point numbers
  - Worst case: % | Average: %

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n} (y_i - \hat{y})^2}{n}} \qquad NRMSE = \frac{RMSE}{y_{max} - y_{min}}$$

https://www.marinedatascience.co/blog/2019/01/07/normalizing-the-rmse/

REPORT TEMPLATE

- Use the Overleaf; they have a template
- **IEEE Journal Paper Template**Official
  - This is a skeleton file demonstrating the use of IEEEtran.cls with an IEEE journal paper. To start writing your manuscript in Overleaf, simply click the 'Open as template' button above. Additional IEEE templates are also available - please use the tags below to view. These include: additional article templates for specific journals (e.g. IEEE Photonics), templates for conference papers, and user-submitted examples and adaptations.
  - Michael Shell
- https://www.overleaf.com/gallery/tagged/ieee-official

WHAT TO INCLUDE IN THE MID-REPORT

- Elaboration and figures on the Datapath block diagram
- Elaboration and figures on the ASM chart for the controller
- Elaboration and figures on post-layout netlist functional verification methods and results
- Elaboration and figures of post-synthesis primetime reports for setup and hold slacks and power consumption (e.g., report screenshot)
- Elaboration and figures on FFT output in FP32 from Python or Matlab simulation; magnitude and phase vs frequencies plots
- Elaboration and figures on FFT output from post-synthesis netlist; magnitude and phase vs frequencies plots
- Elaboration and Table for the six metrics in the slides #4-5

WHAT TO INCLUDE IN THE FINAL REPORT

- Include everything from the mid-report
- Elaboration and figures on post-layout netlist functional verification methods and results
- Elaboration and figures on the layout (e.g., screenshots from Cadence Virtuoso with annotations)
- Elaboration and figures on the LVS and DRC results (e.g., screenshots from Calibre)
- Elaboration and figures on post-layout primetime reports for setup and hold time slacks and power consumption (e.g., report screenshot)
- Elaboration and figures on FFT output from post-layout netlist; magnitude and phase vs frequencies plots
- Elaboration and Table for the six metrics in the slides #4-5

- (Bonus 10 pts) Elaboration and figures on the post-layout SPICE-level power and performance verification methods and results; Use either FINESIM or HSPICE

PAPER TO CONSULT

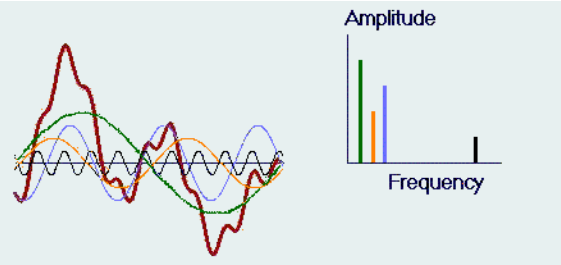
- Joao Pedro Cerqueira, Mingoo Seok, "0.17mm^2 3.19nJ/Transform 256-pt FFT Processor based on Spatiotemporal Active Leakage Suppression Techniques," *European Solid-State Circuits Conference (ESSCIRC)*, 2017
- Uploaded in the Courseworks

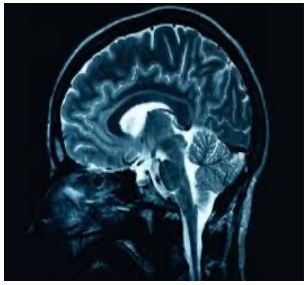
FFT Hardware

Ref: Parhi Chap. 8 and Frerking Chap. 2

FAST FOURIER TRANSFORM (FFT)

- Discrete Fourier Transform (DFT) transforms an input sequence of sampled data (a signal) from the time (or space) domain to the frequency domain. The signal form is described as a sum of sine or cosine waves of different frequencies, phases, and amplitudes.
- Used in diverse science and engineering applications such as communications, sensing, astronomy, optics, geology, medical imaging, biology, physics, chemistry, and genetics.
- Digital audio processing, image processing, spectrum analysis, filter design, analyzing cosmic signals, analyzing seismic signals, analyzing DNA sequences, orthogonal frequency-division multiplexing (OFDM), and more.





FAST FOURIER TRANSFORM (FFT)

- Fast/computationally-efficient algorithm to compute Discrete Fourier Transform (DFT).
  - FFT and DFT produce the **same** results.
  - Complexity of FFT vs DFT: $O(N*\log_2(N))$ vs $O(N^2)$

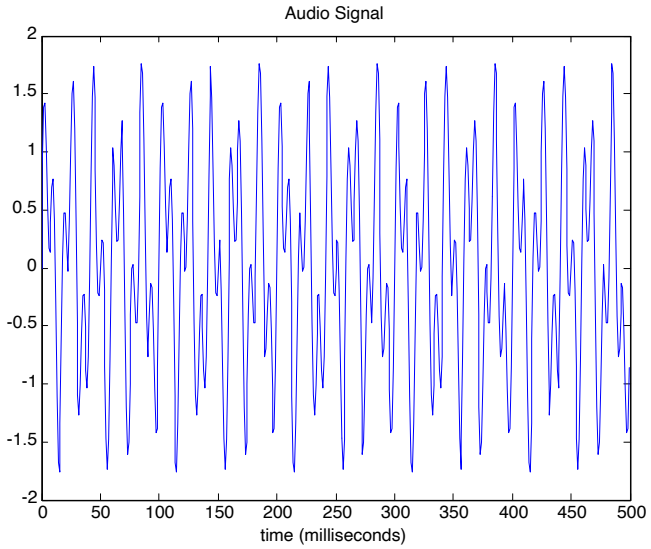
FFT SAVINGS

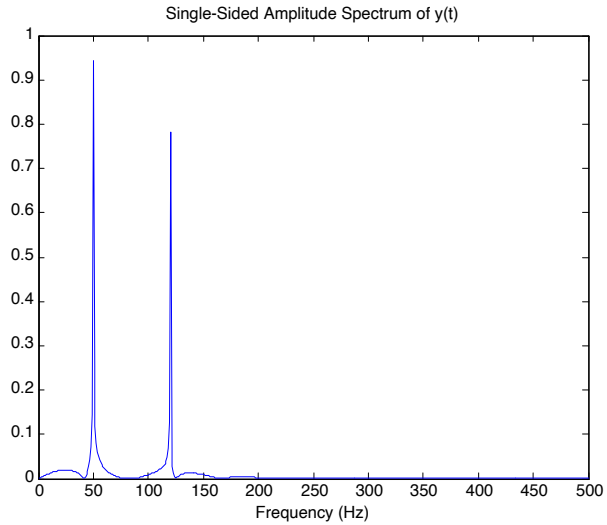
N	DFT operation count, O((N-1)^2)	FFT operation count, O(N/2*log2(N))
8	49	12
16	225	32
32	961	80
64	3969	192
128	16129	448
256	65,025	1024
1024	1,046,529 (~1M)	5120 (~5K)
4096	16,769,025 (~17M)	24576 (~25K)

As N becomes larger, the savings of FFT over DFT increase. For example, when N=1024, 200X reduction is expected

AUDIO ANALYSIS

Consider data sampled at **1000 Hz**. Form a signal containing a **50-Hz** sinusoid of amplitude 1 and **110-Hz** sinusoid of amplitude 0.8. Signal is shown below on left.





It is easy to identify the frequency components by looking at the DFT/FFT of the signal.

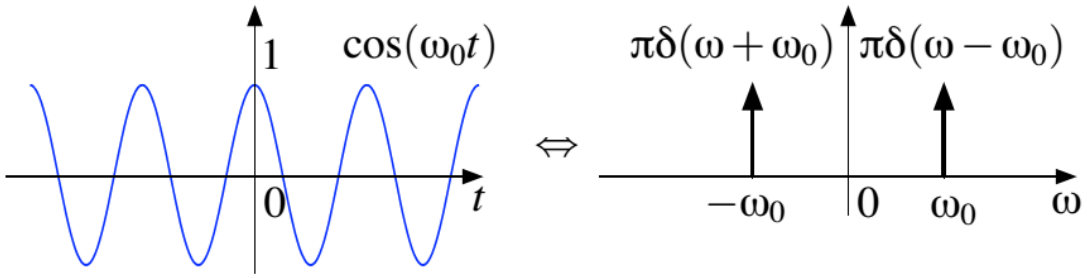
(CONTINUOUS) FOURIER TRANSFORM

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft}\, dt,$$
$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft}\, df.$$

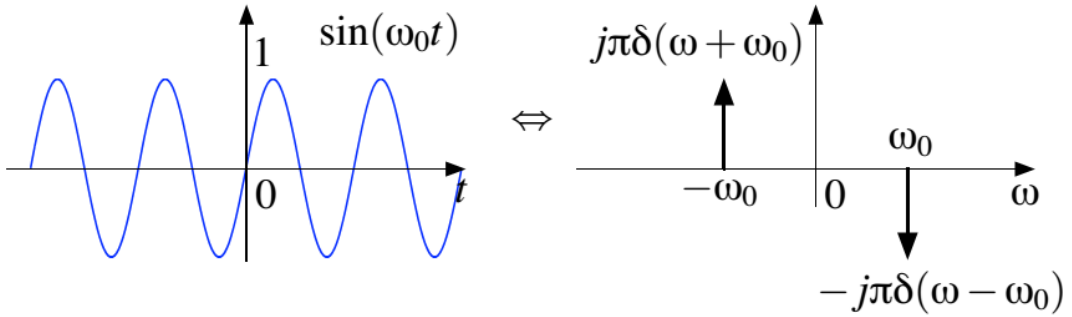
- Two applications
  - Examine the frequency content of a signal
  - Convolution in the time domain can be performed as multiplication in the frequency domain

(CONTINUOUS) FOURIER TRANSFORM

$$\cos(2\pi f_0 t) \Leftrightarrow \frac{1}{2}\left(\delta(f - f_0) + \delta(f + f_0)\right)$$



$$\sin(f_0 t) \Leftrightarrow \frac{j}{2}\left(\delta(f + f_0) - \delta(f - f_0)\right)$$



DISCRETE FOURIER TRANSFORM (DFT)

The **N-point** complex Discrete Fourier Transform (DFT) X for an **N-point** complex input signal x is given by

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}$$

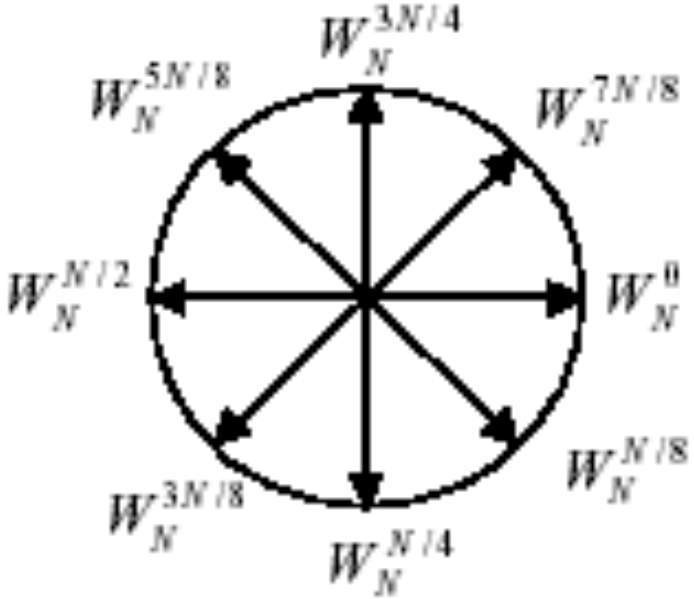
Where k=0,1,…N-1 and the twiddle (or rotation or phase) factor is given by

$$W_N^{nk} = e^{-j2\pi\frac{nk}{N}}$$

$$e^{-j2\pi\frac{nk}{N}} = \cos\left(2\pi\frac{nk}{N}\right) - j\cdot\sin\left(2\pi\frac{nk}{N}\right)$$

So each twiddle factor has an amplitude of 1 and a rotation. So each is a vector in a unit circle.

TWIDDLE FACTOR



$$W_N^{k+N/2} = -W_N^k; \qquad W_N^{k+N} = W_N^k$$

$$W_N^0 = -W_N^{N/2} = 1$$

$$W_N^{N/4} = -W_N^{3N/4} = -j$$

$$W_N^{N/8} = -W_N^{5N/8} = \frac{\sqrt{2}}{2}(1-j)$$

$$W_N^{3N/8} = -W_N^{7N/8} = \frac{\sqrt{2}}{2}(1+j)$$

$$(a+jb)*W_8^1 = \frac{\sqrt{2}}{2}[(a+b)+j(b-a)]$$

$$(a+jb)*W_8^3 = \frac{\sqrt{2}}{2}[(b-a)-j(b+a)]$$

MEANING OF N & K IN DFT

- N: the length of DFT
  - Larger N gives higher resolution results in the frequency domain
- k = 0, 1, 2, … N-1 : represents a frequency
  - Frequency = k*fs/N
  - Where fs is the sampling frequency
- Example
  - A signal whose relevant frequency band is: 22,050 Hz
  - fs = 44,100 Hz, based on the Nyquist-sampling theorem
  - If 1024-point DFT is performed for the signal:
    - The frequency band is divided into N/2, or 512 bin
    - The first bin, i.e., k=0, is DC
    - The second bin is 1*fs / N, or 44100/1024 = 43 Hz
    - The frequency resolution = 43 Hz

DFT AS MATRIX-VECTOR PRODUCT

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & W_N^1 & W_N^2 & W_N^3 & \cdots & W_N^{N-1} \\ 1 & W_N^2 & W_N^4 & W_N^6 & \cdots & W_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & W_N^{3(N-1)} & \cdots & W_N^{(N-1)(N-1)} \end{bmatrix} * \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ \vdots \\ x[N-1] \end{bmatrix}$$

$$X \qquad\qquad W_N \qquad\qquad * \quad x$$

- If we compute DFT directly using the above equation, we need $(N-1)^2$ multiplications $N(N-1)$ additions. Complexity is $\sim O(N^2)$
- Is it possible to reduce the computation effort ?
- See that W_N matrix is symmetric
- We will use recursion and properties of W_N to simplify the computations. [Cooley-Tukey- 1965; Gauss 1805]

DFT TO FFT

$$X[k]=\sum_{n=0}^{N-1}x[n]W_N^{nk}, \text{ where } W_N^{nk}=e^{-j2\pi\frac{nk}{N}}$$

Separate DFT terms for even and odd sequences

$$X[k]=\sum_{r=0}^{\frac{N}{2}-1}x[2r]W_N^{2rk}+\sum_{r=0}^{\frac{N}{2}-1}x[2r+1]W_N^{(2r+1)k} \quad \rightarrow \quad X[k]=\sum_{r=0}^{\frac{N}{2}-1}x[2r]W_N^{2rk}+W_N^k\sum_{r=0}^{\frac{N}{2}-1}x[2r+1]W_N^{2rk}$$

$$X[k]=\sum_{r=0}^{\frac{N}{2}-1}x[2r]W_{N/2}^{rk}+W_N^k\sum_{r=0}^{\frac{N}{2}-1}x[2r+1]W_{N/2}^{rk}, \text{ where } 0\le k\le N-1$$

$$W_N^2=e^{j(-\frac{2\pi}{N}2)}=e^{j(-\frac{2\pi}{N/2})}=W_{N/2}$$

DFT TO FFT

$$X[k] = \sum_{r=0}^{\frac{N}{2}-1} x[2r] W_{N/2}^{rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_{N/2}^{rk}, \text{ where } 0 \le k \le N-1$$

Define G and H as N/2 point DFT for even and odd input (x) sequences

$$G[k] = \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{rk} \qquad H[k] = \sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{rk} \qquad 0 \le k \le N/2 - 1$$

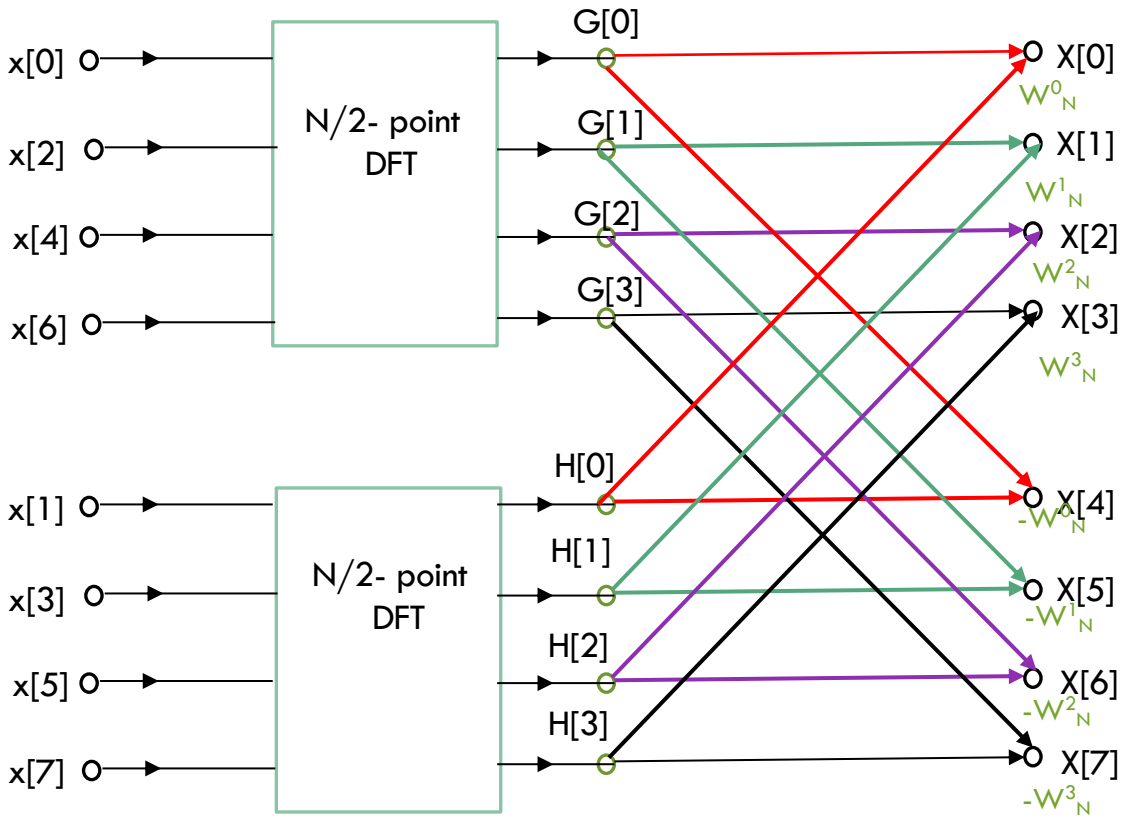
N-point DFT is now based on two N/2-point DFTs

$$X[k] = G[k] + W_N^k H[k] \qquad 0 \le k \le N/2 - 1$$

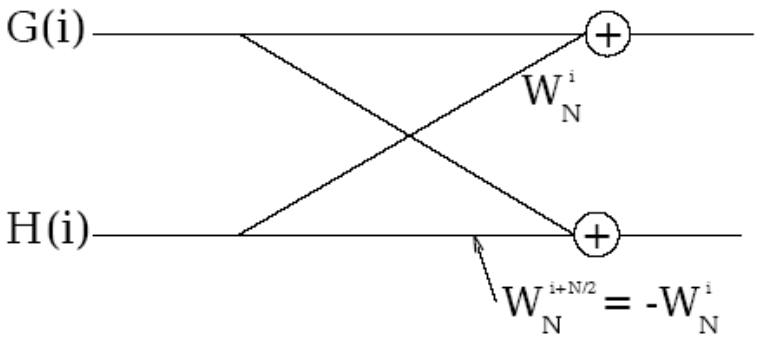
$$X[k + N/2] = G[k + N/2] + W_N^{k+N/2} H[k + N/2] = G[k] - W_N^k H[k] \qquad 0 \le k \le N/2 - 1$$

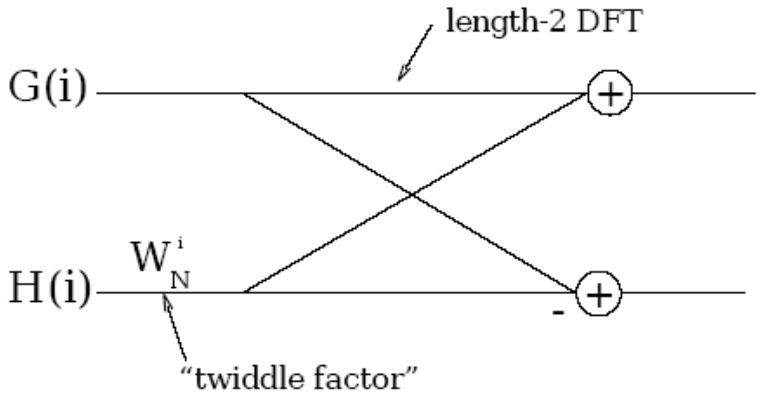
$$W_N^{k+N/2} = -W_N^k, \; G[k + N/2] = G[k], \; H[k + N/2] = H[k]$$

RADIX-2 8-PT FFT, DECIMATION IN TIME



SIMPLIFYING TWIDDLE FACTOR





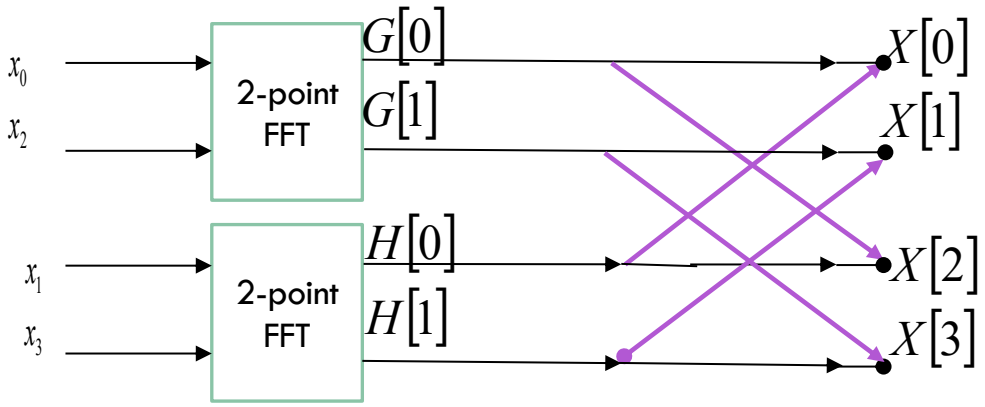
- Two multiplications per BF is not necessary
- After this simplification, the BF becomes, in fact, the BF of 2-pt DFT

FFT, N=4

Define G and H as 2-point FFT for even & odd sequences

$$G = \begin{bmatrix} G[0] \\ G[1] \end{bmatrix} = FFT\left[\begin{bmatrix} x[0] \\ x[2] \end{bmatrix}\right] = W_2 \begin{bmatrix} x[0] \\ x[2] \end{bmatrix} = \begin{bmatrix} x[0]+x[2] \\ x[0]-x[2] \end{bmatrix}$$

$$H = \begin{bmatrix} H[0] \\ H[1] \end{bmatrix} = FFT\left[\begin{bmatrix} x[1] \\ x[3] \end{bmatrix}\right] = W_2 \begin{bmatrix} x[1] \\ x[3] \end{bmatrix} = \begin{bmatrix} x[1]+x[3] \\ x[1]-x[3] \end{bmatrix}$$



4-point FFT for the sequence (x) now can be ***restructured*** in terms of two 2-point DFTs.

$$X[k] = G[k] + W_4^k H[k] \qquad 0 \le k \le 1$$

$$X[k+2] = G[k] - W_4^k H[k] \qquad 0 \le k \le 1$$

$$X = \begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} G[0]+H[0] \\ G[1]-iH[1] \\ G[0]-H[0] \\ G[1]-(-iH[1]) \end{bmatrix}$$

FFT, N=2

Consider the case of N=2

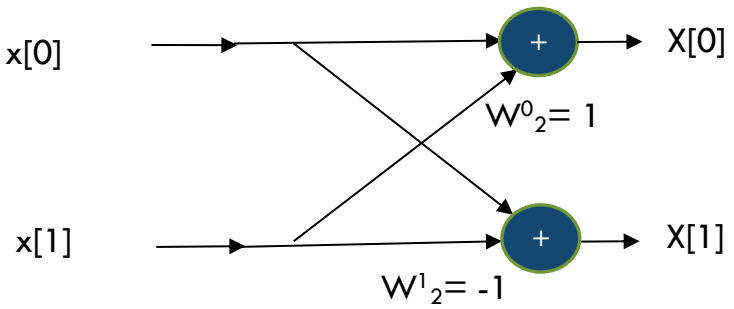
$$W_2^{nk} = e^{-j2\pi\frac{nk}{2}}$$

$$e^{-j2\pi\frac{nk}{2}} = \cos\left(2\pi\frac{nk}{2}\right) - j\cdot\sin\left(2\pi\frac{nk}{2}\right)$$

$$W_2 = \begin{bmatrix} 1 & 1 \\ 1 & W_2^1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$X = \begin{bmatrix} X[0] \\ X[1] \end{bmatrix} = W_2 \begin{bmatrix} x[0] \\ x[1] \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \end{bmatrix} = \begin{bmatrix} x[0]+x[1] \\ x[0]-x[1] \end{bmatrix}$$

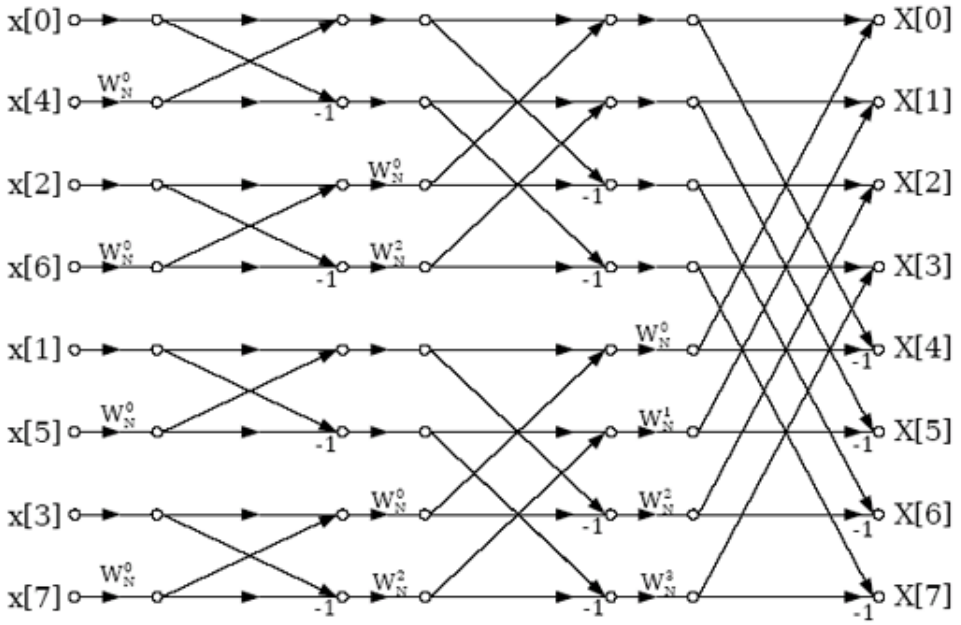
Twiddle Factor $W_2^{nk} = e^{-j2\pi\frac{nk}{2}}$



Needs only 2 (complex) additions. Remember that x[0] and x[1] could be complex numbers.

No multiplications ➔ Good!

RADIX-2 8-POINT DECIMATION-IN-TIME FFT



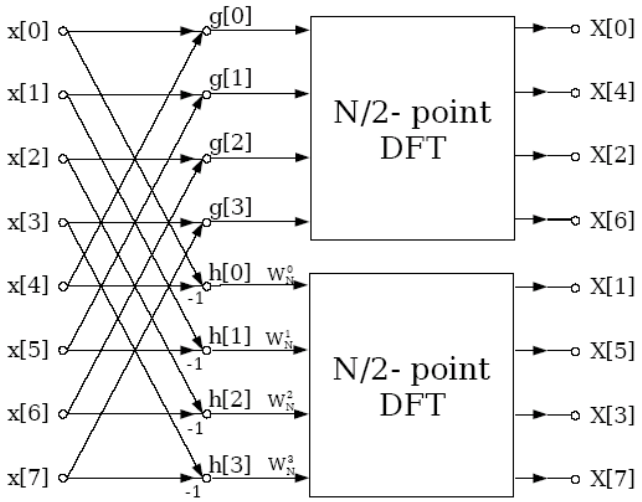
- This algorithm reduces the computational complexity by recursively decomposing input time sequence into smaller sequences (2 N/2 sequence; then 4 N/4 sequences etc, until the smaller sequence length reaches 2)
- $N/2*\log_2 N$ multiplications and $N*\log_2 N$ additions → complexity ~ $O(N*\log_2 N)$

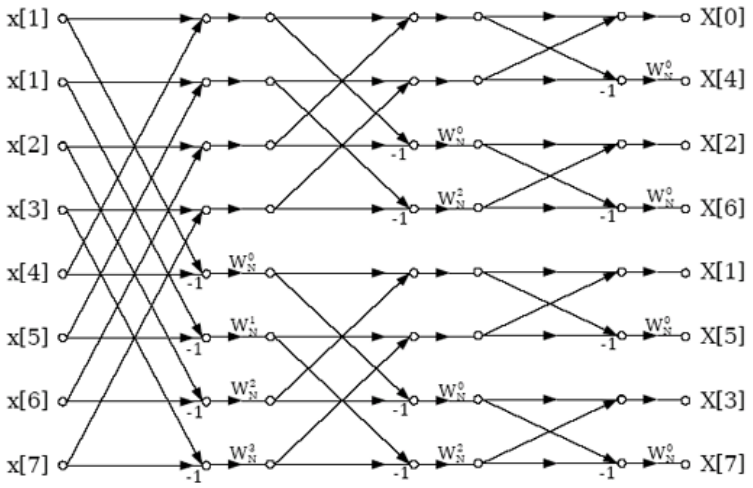
CONTROL FOR 8-POINT DIT FFT

In-order index	In-order index in b	Bit-reverse binary	Bit-reversed index
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

- The bit-reverse index simplifies the controller design

8-PT DECIMATION-IN-FREQ. FFT





- Input is ordered and the output is bit-reverse ordered

RADIX-4 ARCHITECTURE

- N-point FFT is calculated of *four N/4* DFTs, instead of two N/2 DFTs
- Only support the FFT with $N = 4^v$

- ☹ Butterfly is more complex
- ☺ Overall more efficient in terms of computation complexity

RADIX-4 ARCHITECTURE

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk}$$

$$= \sum_{n=0}^{N/4-1} x(n) W_N^{nk} + \sum_{n=N/4}^{2N/4-1} x(n) W_N^{nk} + \sum_{n=2N/4}^{3N/4-1} x(n) W_N^{nk} + \sum_{n=3N/4}^{N-1} x(n) W_N^{nk}$$

$$= \sum_{n=0}^{N/4-1} x(n) W_N^{nk} + \sum_{n=0}^{N/4-1} x\left(n+N/4\right) W_N^{(n+N/4)k} + \sum_{n=0}^{N/4-1} x\left(n+N/2\right) W_N^{(n+N/2)k}$$
$$+ \sum_{n=0}^{N/4-1} x\left(n+3N/4\right) W_N^{(n+3N/4)k}$$

$$= \sum_{n=0}^{N/4-1} \left[\begin{array}{l} x(n) + x\left(n+N/4\right) W_N^{(\ N/4)k} + x\left(n+N/2\right) W_N^{(\ N/2)k} \\ + x\left(n+3N/4\right) W_N^{(\ 3N/4)k} \end{array} \right] W_N^{nk} \qquad (1)$$

[TI; SPRA152 by C. Wu, 1998]

RADIX-4 ARCHITECTURE

$$W_N^{(N/4)k} = \left[\cos\left(\pi/2\right) - j\sin\left(\pi/2\right)\right]^k = (-j)^k$$

$$W_N^{(N/2)k} = \left[\cos(\pi) - j\sin(\pi)\right]^k = (-1)^k$$

$$W_N^{(3N/4)k} = \left[\cos\left(3\pi/2\right) - j\sin\left(3\pi/2\right)\right]^k = j^k$$

$$X(k) = \sum_{n=0}^{N/4-1} \left[\begin{array}{l} x(n) + (-j)^k x\left(n + N/4\right) + (-1)^k x\left(n + N/2\right) \\ +(j)^k x\left(n + 3N/4\right) \end{array} \right] W_N^{nk} \qquad (2)$$

[TI; SPRA152 by C. Wu, 1998]

RADIX-4 ARCHITECTURE

let $W_N^4 = W_{N/4}$.

$$X(4k) = \sum_{n=0}^{N/4-1} \left[\begin{array}{l} x(n) + x\left(n + N/4\right) + x\left(n + N/2\right) \\ + x\left(n + 3N/4\right) \end{array} \right] W_{N/4}^{nk}$$

$$X(4k+1) = \sum_{n=0}^{N/4-1} \left[\begin{array}{l} x(n) - \mathrm{j}x\left(n + N/4\right) - x\left(n + N/2\right) \\ + \mathrm{j}x\left(n + 3N/4\right) \end{array} \right] W_N^n W_{N/4}^{nk}$$

$$X(4k+2) = \sum_{n=0}^{N/4-1} \left[\begin{array}{l} x(n) - x\left(n + N/4\right) - x\left(n + N/2\right) \\ - x\left(n + 3N/4\right) \end{array} \right] W_N^{2n} W_{N/4}^{nk}$$

$$X(4k+3) = \sum_{n=0}^{N/4-1} \left[\begin{array}{l} x(n) + \mathrm{j}x\left(n + N/4\right) - x\left(n + N/2\right) \\ - \mathrm{j}x\left(n + 3N/4\right) \end{array} \right] W_N^{3n} W_{N/4}^{nk}$$

for $k = 0$ to $N/4 - 1$

[TI; SPRA152 by C. Wu, 1998]

RADIX-4 ARCHITECTURE

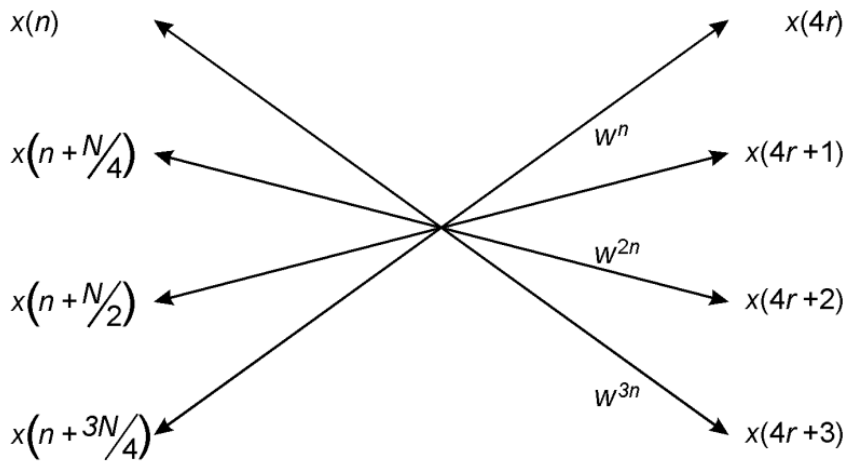
$$X(4k) = \sum_{n=0}^{N/4-1} \left[\begin{matrix} x(n) + x\left(n+N/4\right) + x\left(n+N/2\right) \\ +x\left(n+3N/4\right) \end{matrix} \right] W_{N/4}^{nk}$$

$$X(4k+1) = \sum_{n=0}^{N/4-1} \left[\begin{matrix} x(n) - \mathrm{j}x\left(n+N/4\right) - x\left(n+N/2\right) \\ +\mathrm{j}x\left(n+3N/4\right) \end{matrix} \right] W_N^n W_{N/4}^{nk}$$

$$X(4k+2) = \sum_{n=0}^{N/4-1} \left[\begin{matrix} x(n) - x\left(n+N/4\right) - x\left(n+N/2\right) \\ -x\left(n+3N/4\right) \end{matrix} \right] W_N^{2n} W_{N/4}^{nk}$$

$$X(4k+3) = \sum_{n=0}^{N/4-1} \left[\begin{matrix} x(n) + \mathrm{j}x\left(n+N/4\right) - x\left(n+N/2\right) \\ -\mathrm{j}x\left(n+3N/4\right) \end{matrix} \right] W_N^{3n} W_{N/4}^{nk}$$

for $k = 0$ to $N/4 - 1$

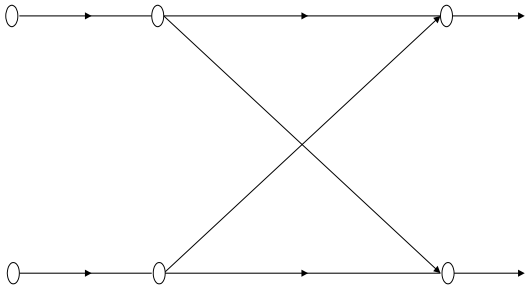


$X(4k)$, $X(4k+1)$, $X(4k+2)$ and $X(4k+3)$ are $N/4$ -point DFTs. Each of their $N/4$ points is a sum of four input samples $x(n)$, $x(n+N/4)$, $x(n+N/2)$ and $x(n+3N/4)$, each multiplied by either +1, -1, j, or -j. The sum is multiplied by a twiddle factor (W_N^0, W_N^n, W_N^{2n}, or W_N^{3n}).

[TI; SPRA152 by C. Wu, 1998]

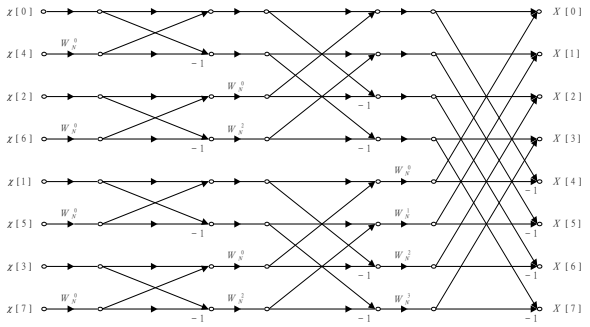
HARDWARE ARCHITECTURE

Reuse Single Butterfly



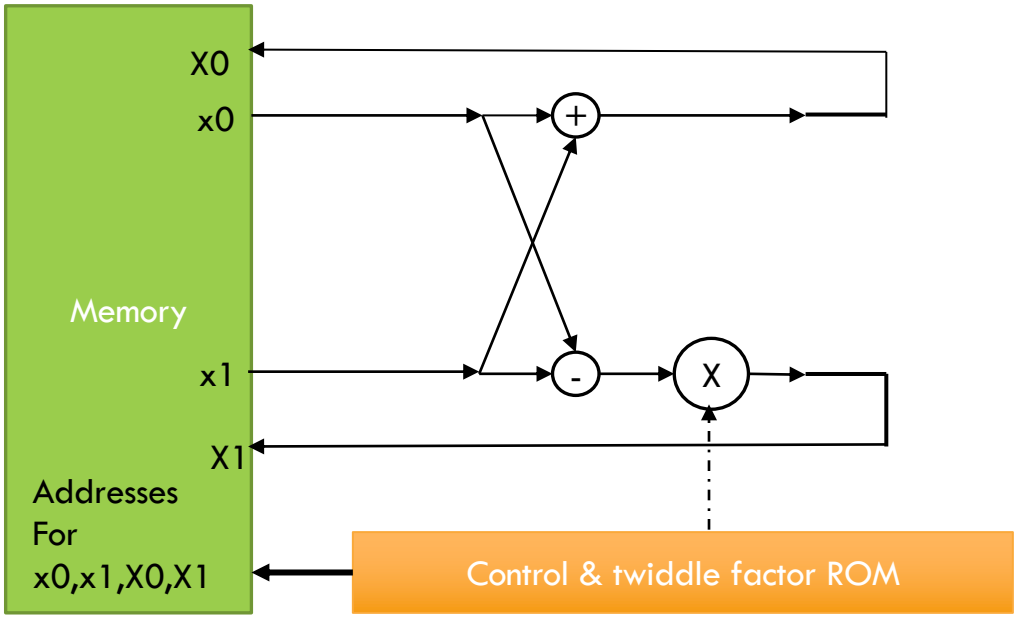
- Slow
- Small
- More complicated control

Fully Spread



- Fast
- Large
- Simple control (embedded in the HW)

SINGLE BF ARCHITECTURE



- Single Butterfly. One butterfly operation per clock.
- Flexible and can support different FFT parameters.
- An N-point FFT requires N/r $\log_r N$ radix-r butterfly computations and 2N $\log_r N$ R/W RAM access
- Improvements: use cache to store the frequently used data

MULTIPLE BUTTERFLIES

- Use multiple, e.g, N/2 BFs
  - Higher throughput: slow by a factor of $\log_2 N$ as compared to a 2-D array.
  - Need to take care about the complex communication structure
  - BFs do not have fixed coefficients. They need to change after each cycle